C++, an IDE that's often called MSVC. This is, of course, very powerful and has a good library associated with it. MSVC also allows the user to develop applications by using Windows system calls. But there is a major problem with the MSVC compiler as far as an open source enthusiast is concerned. Even though MSVC is available as freeware and trial ware, it is essentially a proprietary software, making it anathema to the open source community. Even if this aspect of MSVC is overlooked as being just a philosophical argument, there is an even bigger problem with MSVC and C programming. MSVC only supports an older version of C called ANSI C (also known as C89). The newer versions of C like C99 and C11 are not supported, nor are there any immediate plans to remedy this situation. The lack of support for C99 and C11 is especially crippling to students of C who want to work with the latest versions of the language.

MinGW

If MSVC has so many drawbacks as a C learning tool on Windows, what other alternatives do we have? There is Cygwin, a POSIX-compatible environment that runs on Windows. But in this article we will discuss MinGW for programming in C on Windows. MinGW (Minimalist GNU for Windows) is a free and open source software development environment to create Windows applications. It is licensed under the GNU General Public License. MinGW includes a simplified port of the GNU Compiler Collection (GCC), a compiler system for Linux. It also provides developer utilities for Windows like an assembler, linker, etc. Freely distributable Windows-specific header files and libraries that support the use of the Windows application programming interfaces (API) are also provided by MinGW.

Since both Cygwin and MinGW can be used to execute C programs on Windows, let's look at the differences between the two and why we ought to choose MinGW over Cygwin. The motto of Cygwin, which is 'Get that Linux feeling on Windows', clearly tells us its intended purpose. Cygwin helps users to port Linux applications to Windows. It acts as an intermediate layer between Linux applications and the Windows operating system. So, it is theoretically possible to run any Linux application on Windows using Cygwin, including GCC (the compiler systems of Linux).

So if GCC can be installed and executed on Windows using Cygwin, why should we use MinGW? The first reason is that installing GCC over Cygwin is relatively difficult and prone to errors, whereas the installation of MinGW on Windows is very easy. The second reason not to install GCC on Cygwin is that you need to work with terminal commands to execute the program, whereas it is easy to install and use an IDE on top of MinGW, simplifying the overall program development. The third reason is that it would be an overkill. If your intention is to program in C on Windows, all you need is MinGW and some IDE. Cygwin is much more powerful and is intended for a lot of other purposes too, executing C programs on Windows being just one among them. On a personal note, I believe Linux should be experienced on a Linux operating system and not on Windows using Cygwin. The fourth reason for not using GCC and Cygwin to execute C programs is that by using them, you are developing applications for the Linux operating system on Windows, whereas by using MinGW, you are developing applications for the Windows operating system on Windows itself. The point is that if you are planning to develop applications for Linux, the best platform is Linux itself.

To illustrate this final difference further, let us execute the following two simple C programs in both MingW and Cygwin. To execute the C programs with MinGW, we use the popular IDE called Code::Blocks, about which more will be shared later in this article. As mentioned earlier, to execute the program in Cygwin, we use a version of GCC, modified for use on Cygwin. Both the C programs use the library function *system()* to use system calls and commands of Linux and Windows.

The first program named *pgm1.c* is given below. It tries to execute the commands *cd* and *date* available in both Linux and Windows. Please note that the behaviour of the commands *cd* and *date* is different in Linux and Windows. Readers will understand this clearly after executing the programs with MinGW and Cygwin.

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    system("cd");
    system("date");
    return 0;
}
```

The second program named *pgm2.c* is also given below. This tries to execute the Linux command *pwd* to print the name of the current working directory and the command *date* available in both Linux and Windows.

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    system("pwd");
    system("date");
    return 0;
}
```

Figure 1 shows the output of the program *pgm1.c* when executed with Code::Blocks and MinGW. The output shows that both the commands were executed properly. Since *cd* is a valid Windows command, the name of the current working directory is printed. Please note that the full path is printed in this case. The *date* command in Windows is used to print the current date and then modify it. If you look closely at the output, you will see that the *date* command is treated as a Windows command by MinGW; so it prints the current date in the Windows format and